Apuntes · Clase 30 · Preparación para Machine Learning

Para leer antes de los ejercicios · lectura de ~35 min

Lo más importante, en 6 ideas

1. **Tipos y calidad.** Cada variable tiene un tipo (numérica o categórica) y los datos se juzgan por exactitud, completitud, coherencia y frescura.
2. **Imputar con criterio.** Media (simétrica), mediana (atípicos), moda (categórica); siempre aprendida con el entrenamiento.
3. **Tres conjuntos.** Entrenamiento aprende, validación elige, prueba da el veredicto final una sola vez.
4. **Regularizar y estandarizar.** Ridge achica y Lasso puede anular; el castigo depende de las unidades, por eso se estandariza.
5. **K-Fold y estratificación.** Un promedio y una dispersión en vez de una partición suerte; la clase rara se reparte proporcionalmente.
6. **Métricas y fuga.** Precision, recall y F1 según el costo del error; todo el preprocesamiento va dentro de un Pipeline.

Cada sección trae una explicación, un **ejemplo resuelto con números distintos** a los de los ejercicios y una **pista** para el notebook `C30_PreparacionML.ipynb` o el cuaderno `C30_Ejercicios_Cuaderno_PreparacionML.pdf`.

1 Tipos de variables y one-hot

Una variable **numérica** se puede sumar y promediar (continua: superficie; discreta: número de hijos). Una **categórica** son etiquetas: **nominal** si no hay orden (región) y **ordinal** si lo hay (básico, medio, alto). Los modelos solo reciben números, así que las categóricas se convierten con **one-hot**: una columna 0/1 por categoría.

Ejemplo resuelto · clientes de un seguro

Columna	Valores	Tipo
edad	23, 41, 67	numérica (se promedia)
n_hijos	0, 1, 3	numérica discreta
region	Norte, Sur, Este	categórica nominal → one-hot: 3 columnas 0/1
nivel_estudios	básico, medio, superior	categórica ordinal (hay orden)
tiene_deuda	Sí, No	categórica binaria → una columna 0/1

Con one-hot, `region` deja de ser una columna y pasa a tres: `region_Norte`, `region_Sur`, `region_Este`. Un cliente del Sur queda como (0, 1, 0).

Pista para la Parte A del notebook y el Bloque 1 del cuaderno

Pide `viviendas.dtypes` y clasifica cada columna; después aplica `pd.get_dummies` a `barrio` y cuenta las columnas nuevas.

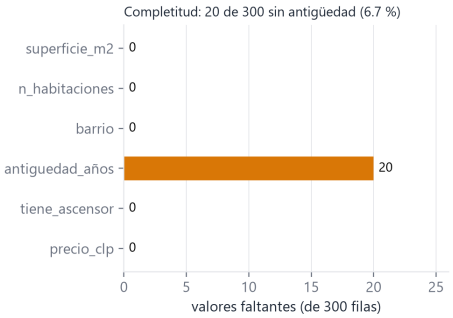
2 Calidad de datos e imputación

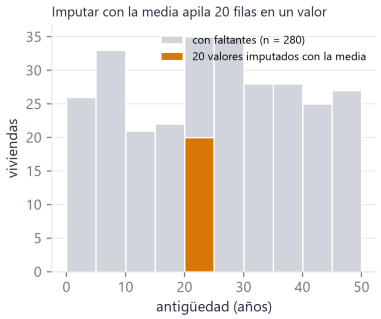
Un dato faltante se puede completar con un estadístico. **Media** si la variable es simétrica y sin atípicos; **mediana** si hay atípicos o asimetría; **moda** si es categórica. Si el faltante es muy grande (un tercio), no se imputa: se revisa cómo se recogieron los datos. Imputar con la media deja la **desviación estándar más pequeña**, porque todos los rellenos caen en el mismo valor.

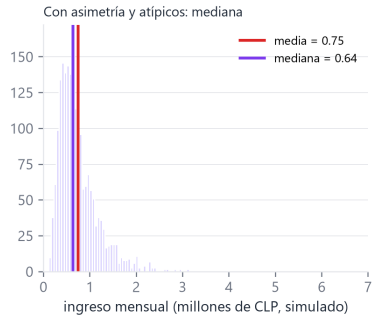
Ejemplo resuelto · seis clientes, un ingreso faltante

Pregunta	Cálculo	Resultado
Ingresos (M CLP): 1.2, 1.4, 1.5, 1.6, 9.0, falta · media de los 5 observados	(1.2 + 1.4 + 1.5 + 1.6 + 9.0)/5 = 14.7/5	2.94
Mediana de los 5 observados	valor central ordenado	1.5
Región: Norte, Sur, Sur, Este, Sur, falta · moda	la que más se repite (3 de 5)	Sur
Edad: 30, 32, 34, 36, 38 (simétrica) · media	170/5	34

El 9.0 empuja la media a 2.94, más del doble de lo que gana la mayoría: la mediana (1.5) representa mejor al cliente típico. Las cuatro dimensiones de calidad: **exactitud** (¿refleja la realidad?), **completitud** (¿falta?), **coherencia** (¿las fuentes concuerdan?) y **frescura** (¿está vigente?).







Pista para la Parte A del notebook y el Bloque 2 del cuaderno

En A2 cuenta los faltantes con `isna().sum()`, imputa con `SimpleImputer` y compara la desviación antes y después. Recuerda: el imputador se ajusta con el entrenamiento.

3 Entrenamiento, validación, prueba y sobreajuste

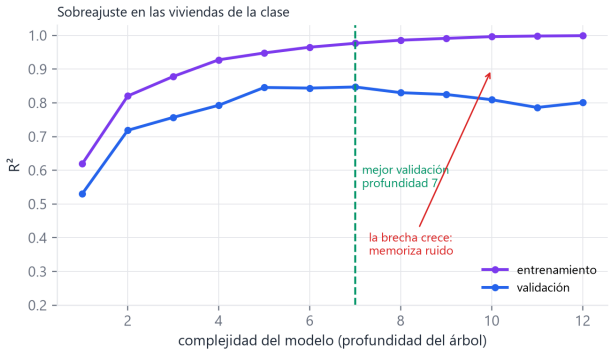
El **entrenamiento** ajusta los parámetros; la **validación** sirve para comparar alternativas (alpha, profundidad); la **prueba** se reserva para el final y se mira una sola vez. El **sobreajuste** aparece como una brecha: rinde mucho mejor en entrenamiento que en datos que no vio.

Ejemplo resuelto · 1.000 registros con una división 60/20/20

Pregunta	Cálculo	Resultado
Tamaños de cada conjunto	0.6·1.000 · 0.2·1.000 · 0.2·1.000	600 · 200 · 200
Cómo hacerlo con train_test_split	test_size = 0.40 y luego test_size = 0.50	600 y 400 → 200 y 200
Modelo A: R^2 0.99 en entrenamiento y 0.70 en validación	brecha = 0.99 − 0.70	0.29 → sobreajuste
Modelo B: R^2 0.86 y 0.82	brecha = 0.04	estable: B generaliza mejor

Se elige B por su validación (0.82 > 0.70), aunque A tenga mejor entrenamiento. La prueba se reserva para informar el modelo ya elegido.





Pista para la Parte B del notebook y el Bloque 3 del cuaderno
Reproduce el 70/15/15 de la profesora (210/45/45) y luego usa la validación para elegir la profundidad de un árbol; mira la prueba solo al final.

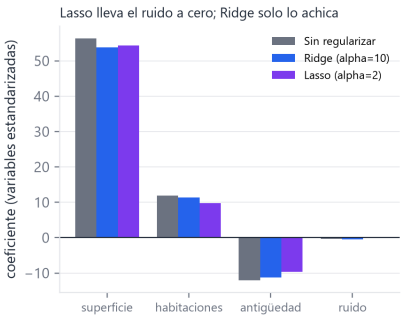
4 Ridge, Lasso y estandarización

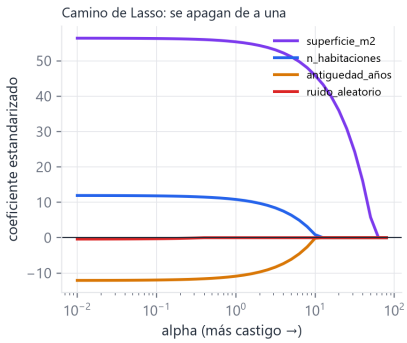
Ridge minimiza el error más **$\alpha \cdot \Sigma \beta^2$** y Lasso el error más **$\alpha \cdot \Sigma |\beta|$**. Como la penalización mira el tamaño de los coeficientes, una variable en unidades pequeñas necesita un coeficiente grande y paga más. **Estandarizar** (z = (x − media)/desviación) deja todo en escala comparable.

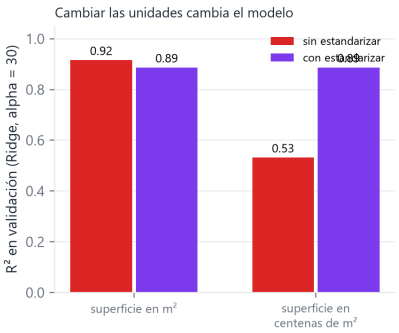
Ejemplo resuelto · un mismo efecto en dos unidades

Pregunta	Cálculo	Resultado
Superficie: media 80 m^2, desviación 20 · z de una casa de 120 m^2	(120 − 80)/20	z = 2
Habitaciones: media 3, desviación 1.5 · z de 4.5	(4.5 − 3)/1.5	z = 1
Efecto de 3 por m^2 frente a 300 por centena de m^2 (penalidad Ridge, α = 1)	3^2 frente a 300^2	9 frente a 90.000
Veces más castigado	90.000 / 9	10.000

Es el mismo efecto con otras unidades, pero la penalización lo castiga 10.000 veces más. Con las viviendas de la clase, Ridge (alpha 30) pasa de R^2 0.916 a 0.532 al cambiar a centenas de m^2; con estandarización queda en 0.887 en ambos casos. Lasso, además, puede llevar coeficientes a cero: en la clase anuló `ruido_aleatorio`.







Pista para la Parte C del notebook y el Bloque 4 del cuaderno
En C1 compara los coeficientes con y sin `StandardScaler` y repite con la superficie en centenas de m^2. Pregunta clave: ¿qué cambia al cambiar las unidades?

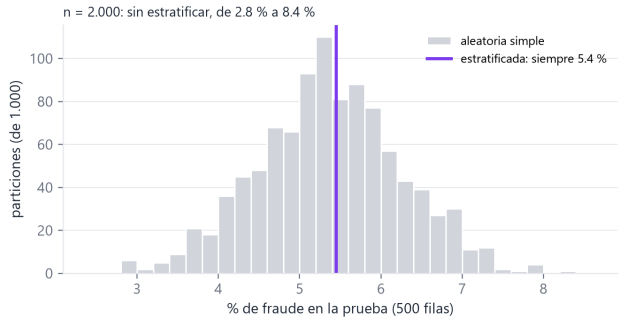
5 K-Fold y estratificación

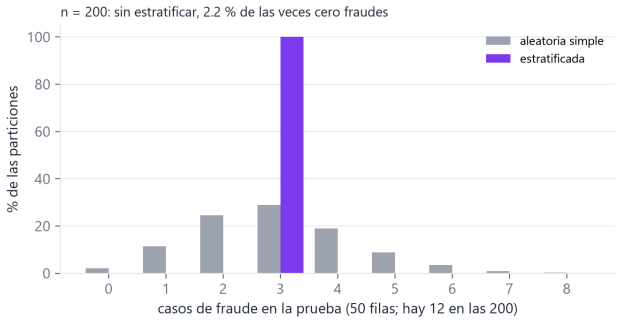
K-Fold divide los datos en K pliegues y usa cada uno una vez como validación. El resultado es un **promedio** y una **desviación entre pliegues**. La **estratificación** reparte las clases en proporción: con 5 % de fraude, la prueba también tiene ≈ 5 %.

Ejemplo resuelto · K = 4 con 100 filas, y una clase rara

Pregunta	Cálculo	Resultado
Filas por pliegue (validación) y de entrenamiento	100/4 · 100 − 25	25 · 75
Puntajes: 0.80, 0.84, 0.78, 0.86 · promedio	(0.80 + 0.84 + 0.78 + 0.86)/4	0.82
Desviación (ddof = 0, como NumPy)	$\sqrt{((0.02^2 + 0.02^2 + 0.04^2 + 0.04^2)/4)} = \sqrt{0.001}$	0.032
400 filas con 5 % de fraude, prueba de 25 % · fraudes en la prueba (estratificada)	20 · 0.25 = 5 de 100 filas	5 casos

Sin estratificar, el número de fraudes en la prueba varía alrededor de 5 (desviación ≈ 1.9), y puede salir 2 o 8. Con la estratificación siempre son 5. En el notebook de la clase: 2.000 casos con 5.45 % de fraude → entrenamiento 5.467 % y prueba 5.400 %.





Pista para la Parte C (K-Fold) y la Parte D (estratificación) del notebook y los Bloques 5 y 6 del cuaderno

Calcula el K-Fold con entrenamiento + validación (no con las 300 filas) y repite 1.000 particiones con y sin `stratify` para ver la dispersión.

6 Matriz de confusión y métricas

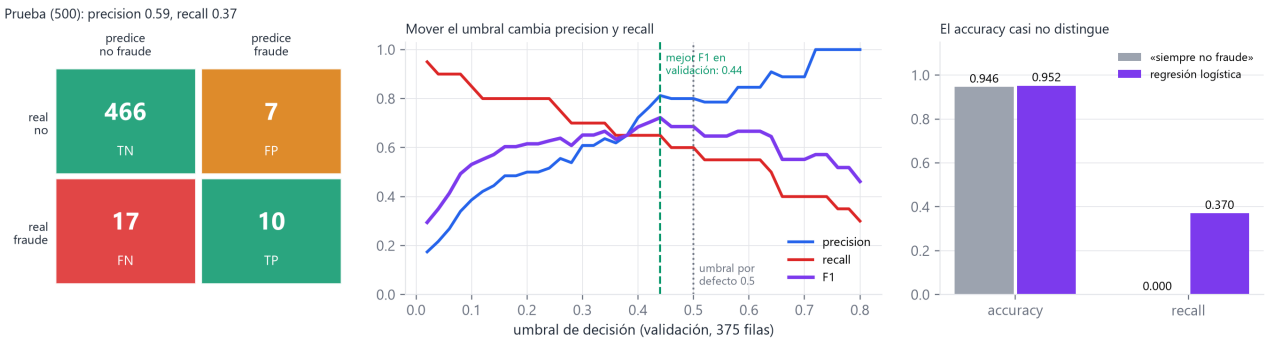
Todo sale de cuatro números. **Precision** = TP/(TP+FP): de lo que marqué como positivo, cuánto lo era. **Recall** = TP/(TP+FN): de los positivos reales, cuántos detecté. Son como probabilidades condicionales: cambia el «total». **F1** es su media armónica. **Accuracy** = (TP+TN)/n puede engañar si una clase domina.

Ejemplo resuelto · 1.000 transacciones, 50 fraudes

TP = 30, FP = 10, FN = 20, TN = 940.

Métrica	Cálculo	Resultado
Accuracy	(30 + 940)/1.000	0.970
Precision	30/(30 + 10)	0.750
Recall	30/(30 + 20)	0.600
F1	2·0.75·0.60/(0.75 + 0.60) = 0.90/1.35	0.667
Accuracy de «siempre no fraude»	950/1.000	0.950

El accuracy 0.970 parece excelente pero el modelo deja escapar 20 de 50 fraudes. Si bajar el umbral sube el recall a 0.8 con más falsas alarmas, el F1 puede mejorar o no: depende del costo de cada error.



Pista para la Parte D del notebook y el Bloque 7 del cuaderno

Calcula las métricas a mano desde `confusion_matrix`, compáralas con scikit-learn y luego mueve el umbral con `predict_proba` usando la validación.

7 Fuga de datos y Pipeline

Un estandarizador ajustado con **todas** las filas conoce la media y desviación de la prueba: eso es fuga. Lo correcto es **fit con entrenamiento** y **transform** en el resto. Un **Pipeline** lo hace solo, y dentro de K-Fold repite el proceso en cada pliegue.

Ejemplo resuelto · un atípico en la prueba

Pregunta	Cálculo	Resultado
Entrenamiento: 10, 12, 14 · media y desviación	media = 12; √((4 + 0 + 4)/3)	12 · 1.633
Prueba: 30 · z con estadísticos del entrenamiento (correcto)	(30 − 12)/1.633	z = 11.0 (atípico claro)
z con estadísticos de las 4 filas (fuga)	media 16.5 · desviación 7.92 → (30 − 16.5)/7.92	z = 1.70 (parece normal)

La fuga ocultó el atípico: el modelo «ya vio» la prueba. Con datos de puro ruido, seleccionar variables con todas las filas dio ≈ 0.79 de exactitud; dentro del Pipeline, ≈ 0.51 (azar).

Quiero…	Código
Escalar y modelar sin fuga	make_pipeline(StandardScaler(), Ridge(alpha=10)).fit(X_train, y_train)
Tratar columnas por tipo	ColumnTransformer([('num', pipe_num, cols_num), ('cat', pipe_cat, cols_cat)])
Validar con Pipeline	cross_val_score(pipe, X_dev, y_dev, cv=KFold(5, shuffle=True, random_state=42))
Evaluar una vez	pipe.fit(X_dev, y_dev); pipe.predict(X_test)

Pista para la Parte B del notebook (B3) y el Bloque 8 del cuaderno

Reproduce el experimento de ruido con y sin Pipeline y explica con una frase por qué difieren.

Lo que debes recordar

- Orden sugerido: apuntes → Bloques 1–4 del cuaderno → Partes A y B del notebook → Bloques 5–8 → Partes C y D → ejercicio de cierre.
- Si te trabas más de 10 minutos, mira la solución, entiéndela y vuelve a hacerlo sin mirar.